HARMONI: **H**ierarchical **AR**chitecture **MO**deling for LLMs with **N**ear/**I**n Memory Computing

Khyati Kiyawat, Yasas Seneviratne, Zhenxing Fan, Morteza Baradaran, Kevin Skadron
University of Virginia
{khyati, yasas, fjy3ws, rgq5aw, skadron}@virginia.edu

***Abstract*—LLM inference has emerged as a strongly memory-bound workload suitable for Processing-in-Memory (PIM) and Processing-near-Memory (PNM) architectures. Yet, most existing PIM-LLM evaluation frameworks are in-house, closed-source, or difficult to extend, and often fail to model end-to-end inference behavior or tensor-allocation and communication effects that critically shape performance. We introduce HARMONI, a fast, modular, memory-centric performance modeling framework designed specifically for hierarchical PIM/PNM architectures running LLM workloads. HARMONI captures the complete inference execution through a task-graph representation and supports user-defined tensor allocation with address-interleaving schemes. It models computation, communication, and queuing delays across logic nodes, enabling realistic mapping of inference kernels onto heterogeneous logic nodes. HARMONI outputs detailed, kernel-wise and resource-wise time and energy breakdowns, providing actionable visibility into architectural bottlenecks. Together, these capabilities make HARMONI a practical tool for rapidly evaluating PIM/PNM designs and DRAM standards (e.g., DDR4, DDR5, GDDR6), enabling systematic co-evaluation of inference workloads and memory-centric architecture designs.**

***Index Terms*—Performance Modeling, Processing-in-Memory, Processing-near-Memory, Hierarchical Architectures**

I. Introduction

LLMs have become a central workload for cloud providers, hyperscalers, and emerging edge AI systems, driving sustained demand for high-throughput, energy-efficient inference infrastructure. During autoregressive decoding, each generated token requires re-reading key–value (KV) cache entries from all preceding tokens across all layers. As models scale, this KV cache traffic grows linearly with sequence length, dominating memory capacity and data transfer cost. KV cache optimization techniques, including quantization, pruning, and eviction, can delay this bottleneck, but do not eliminate it [1]–[5]. Several profiling studies such as [6] find that 95% of the operators are memory bandwidth bound. A large portion of inference time in modern LLMs is thus spent waiting on memory transfers rather than performing arithmetic operations, due to limited memory bandwidth and high data-movement overheads. This imbalance becomes even more pronounced for single-request (batch = 1) decoding, where compute utilization is low but memory traffic remains substantial [7]–[9]. This memory bottleneck naturally creates interest in memory-centric solutions that reduce or localize data movement, such as Processing-in-Memory (PIM) and Processing-near-Memory (PNM). These approaches place compute units closer to where the data resides, allowing the system to exploit DRAM-internal bandwidth and reduce off-chip transfers for memory-intensive operations. As a result, PIM/PNM architectures [10]–[28] have emerged as promising candidates for accelerating the memory-traffic-dominated phases of LLM inference.

However, most of these works rely on in-house simulators to evaluate their proposed architectures. This has two important drawbacks. **Limitation 1:** Among the tools that are open sourced, many are difficult to modify or extend, limiting researchers' ability to explore new architectural ideas or test alternative PIM/PNM designs. **Limitation 2:** For works whose simulators are not publicly available, it becomes challenging to reproduce results or fairly compare against new architectures. The PIM design space involves numerous assumptions about memory organization, compute capability, timing models, and data movement patterns that are often implicit or undocumented, leaving room for error or misinterpretation when evaluating or benchmarking new architectures.

A further limitation of existing PIM/PNM simulators is their prohibitive runtime, leading to **Limitation 3:** Cycle-accurate simulation of LLM inference is inherently slow due to the token-by-token, layer-by-layer execution pattern characteristic of autoregressive decoding. Capturing every memory access, compute operation, and synchronization event at cycle granularity leads to extremely long simulation times. For instance, NeuPIMs [18] uses an ONNX-based simulator whose runtime scales to many hours per decoder layer at moderate token counts. This makes it impractical to explore large design spaces, iterate on different memory organizations, or evaluate end-to-end model execution behavior across diverse workloads.

To address these challenges, we introduce **HARMONI** (**H**ierarchical **AR**chitecture **MO**deling for LLMs with **N**ear/**I**n Memory Computing), a fast and modular design space exploration framework for hierarchical PIM/PNM architectures targeting LLM inference. HARMONI provides a DRAM-aware architecture description that captures channels, ranks, chips, banks, near-memory compute units, and interconnect structures in a unified representation.

HARMONI philosophy. HARMONI is not intended to replace detailed, cycle-accurate simulation frameworks. Instead, it serves as a fast architectural modeling layer that enables rapid exploration of hierarchical PIM/PNM design configurations for diverse LLM inference workloads. HARMONI estimates performance metrics using analytical models for memory access, compute cycles, and inter-node communication, described later in §V. This abstraction enables efficient first-pass analysis of execution behavior and helps identify potential bottlenecks across computation, communication, and memory resources before investing in expensive cycle-accurate simulation. When higher fidelity is required, HARMONI can export traces and system metadata to enable integration with cycle-accurate backend simulators through simulator-specific wrappers or extensions.

While HARMONI is designed to support trace generation for multiple backend simulators, its current implementation provides partial support for compute and network domains. At present, HARMONI can generate compute traces in a format compatible with SCALE-Sim, although users must configure the underlying architectural parameters in SCALE-Sim (e.g., array dimensions, buffer sizes, dataflow) to match the intended target system. HARMONI is also capable of producing network-level traces, but these require wrapper scripts to convert HARMONI's internal trace format into the input formats expected by NoC simulators such as GARNET [29] or BookSim [30]. Finally, because HARMONI tracks the exact row, column, and bank locations of all memory operations, it can in principle generate DRAM-access traces for simulators such as DRAMsim3 [31] and Ramulator 2.0 [32].

In summary, this paper makes the following contributions:

1) We present HARMONI, a modular, memory-centric performance modeling framework for end-to-end LLM inference on hierarchical PIM and PNM architectures.
2) We introduce a hierarchical logic-node abstraction with memory-aware task graphs and tensor placement, enabling systematic exploration of heterogeneous compute placement and DRAM organization.
3) We demonstrate that HARMONI enables fast design space exploration while providing detailed kernel-wise and resource-wise latency and energy breakdowns across data accesses, computation, communication, and queuing delays.

II. Background

A. LLM Inference

Fig. 1 shows a typical autoregressive transformer inference application [33]. The execution comprises two phases: *prefill* (prompt processing) and *decode* (autoregressive generation). During *prefill*, the input prompt is tokenized and each token ID undergoes an embedding lookup to produce an input matrix, which is processed through a stack of decoder layers. The final output is passed to a Language Model Head

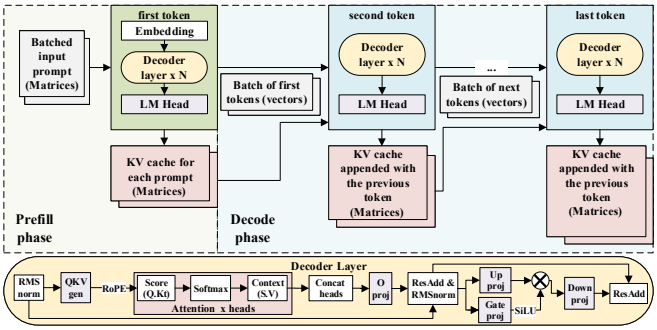

Fig. 1: Transformer Overview (Ex: Llama architecture)

(LM Head) that generates the next token. During *decode*, tokens are generated autoregressively one at a time using the same decoder stack, building and appending Key-Value (KV) cache at each step. The KV cache grows linearly with sequence length, making decode both memory-capacity and bandwidth-bound.

Each decoder layer consists of two sublayers. In the *attention* sublayer, the input is projected into Query (Q), Key (K), and Value (V) vectors across multiple independent heads; past K and V entries are retrieved from the KV cache to compute per-head scores and context vectors. In the *feed-forward network* (FFN) sublayer, the concatenated head outputs pass through output, up, gate, and down projections. Residual connections and RMS normalization are applied around both sublayers. All these layers and sub-layers are sequentially executed per token except the attention heads and algorithmically parallel FFN sublayers within a layer. We can capture such inter-kernel dependencies with a task graph of the application as discussed in §V-C2.

B. Converging Theme in PIM-LLM Accelerators.

Recent work on PIM and PNM architectures for LLM inference exhibits a striking architectural convergence, despite targeting different system contexts (GPU-attached [10]–[13], accelerator-centric [18]–[20], or GPU-free designs [16], [17]). Across these proposals, computation is increasingly organized as a hierarchical PIM/PNM architecture, where different classes of operators are mapped to logic units placed at different levels of the DRAM hierarchy as shown in Fig. 2.

At the lowest level, most architectures introduce bank- or subarray-level compute units optimized for bandwidth-dominated operations, most notably GEMV and Vector operations. For example: SIMD-style MAC units embedded within DRAM banks or bank groups. These units exploit massive DRAM-internal bandwidth and internal parallelism, making them well-suited for KV cache accesses and attention kernels that dominate autoregressive decoding.

At higher levels of the hierarchy such as bank groups, ranks, logic dies, or near-memory controllers, architectures commonly place reduction or special-function

units, including adder trees, softmax, normalization units, or lightweight general-purpose cores. These units aggregate partial results produced by bank-level compute units and execute operations that either require inter-bank communication or complex logic units infeasible to manufacture with DRAM technology. This is where PNM units in the CXL controller, HBM logic die, and DIMM ranks come into the picture.

This hierarchical organization is evident across a broad range of recent proposals. TransPIM [17] combines subarray-level vector operations with near-memory auxiliary compute units (ACUs) to handle reductions and softmax, explicitly introducing bank-to-bank ring broadcast mechanisms to support token-based sharding. AttAcc [10] systematically explores the placement of GEMV units at the bank, bank-group, or logic-die level, while assigning reduction and normalization to higher-level logic. NeuPIMs [18] integrates GDDR6-based PIM with NPU execution and focuses on orchestrating concurrent GEMM and GEMV phases through careful scheduling and micro-architectural support. PAPI [13] differentiates between attention-oriented and fully connected PIM variants, both relying on bank-level GEMV engines with varying throughput configurations. CENT [16] extends this hierarchy further by combining near-bank GEMV-style PIM with near-memory PNM RISC-V cores and supporting collective communication primitives across devices.

Importantly, as PIM architectures mature beyond isolated bank-local execution, there is growing emphasis on inter-unit communication within the memory system. Tensor mapping, partial-sum aggregation, and communication collective operations introduce non-trivial traffic patterns whose performance depends on the topology, bandwidth, and contention behavior of intra-PIM interconnects. As a result, modern PIM/PNM proposals are no longer defined solely by compute placement, but by the interaction between hierarchical compute units and their communication fabric.

Taken together, these trends point to a shared architectural abstraction: hierarchical PIM/PNM systems with distributed compute units, explicit interconnects, and flexible operator placement. Yet, evaluating such architectures requires understanding not only individual kernels, but also how computation, communication, and queuing (scheduling constraints due to structural hazards) effects interact across hierarchy levels under realistic LLM inference workloads.

III. Motivation

A. Template Hierarchical PIM/PNM Architecture

We model a template hierarchical PIM/PNM architecture that places heterogeneous logic units at multiple levels of the memory hierarchy as shown in Fig. 2 and connects them through explicit interconnects. The architecture template comprises four logic-node levels: root, channel, rank, and chip, corresponding to logic units placed at different levels of the memory hierarchy. The *root* node represents either a host processor (CPU/GPU) or a CXL-based switch. We logically partition DRAM ranks into *weight-ranks* and *kv-ranks* to decouple model-weight storage from key–value (KV) cache storage, enabling disaggregated execution of projection and attention kernels. During inference, KV cache entries are generated at *weight-ranks* and subsequently transferred to *kv-ranks* for attention computation. The logic unit at each level is configured with a set of supported operations as mentioned in Tab. I. We assume GEMM computation at chip-level logic unit assuming small systolic arrays that can be integrated into DRAM dies with modest die-area overhead. We treat the GEMM unit as an architectural assumption and leave detailed feasibility analysis to future work. The memory interconnect is modeled as a hierarchical tree topology, where chip nodes connect to rank nodes, rank nodes connect to channel nodes, and channel nodes connect to the root node.

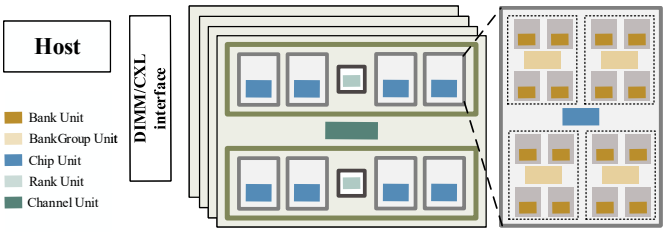

Fig. 2: Representative PIM/PNM architecture for LLMs

TABLE I: Supported Operations by Logic Units

Logic Unit Level	Supported LLM Operations
Chip (*kv-rank*)	FUSED-SCORE, GEMM (GQA), GEMV (MHA)
Chip (*wt-rank*)	APPEND, GEMM, GEMV, LOOKUP, RoPE
Rank	ARGMAX, EWMUL, LOOKUP, RMSNORM, SiLU
Channel	AGGREGATE, CONCAT, RESADD
Root	AGGREGATE, CONCAT, RESADD

We evaluate this template PIM/PNM architecture using HARMONI across a range of DRAM configurations and inference workloads in this paper (§VI).

B. Need for a Memory-Centric Simulation Framework

The hierarchical organization described in §III-A has important implications for how LLM inference systems must be modeled. In these PIM/PNM architectures, both computation and data movement are distributed across multiple levels of the memory hierarchy, with heterogeneous logic units operating under different bandwidth, latency, and communication constraints. As a result, end-to-end inference performance is shaped not only by kernel execution time, but also by tensor placement, inter-level communication, and queuing effects that arise from shared resources.

Existing architectural simulators were not designed around this execution model. Systolic-array and NPU simulators typically assume a small number of homogeneous compute engines connected to a centralized memory system, and treat memory accesses as either idealized bandwidth constraints

TABLE II: Modeling support for hierarchical PIM/PNM LLM inference across simulator abstractions. Entries reflect native support within each simulator's intended scope; "Partial" denotes indirect or limited support.

Requirement for hierarchical PIM/PNM LLM inference	**Kernel-level compute** [34], [35]	**Accelerator-level graph** [36]	**System / serving-level** [37], [38]	**PIM / NDP frameworks** [39], [40]	**HARMONI**
End-to-end LLM execution with inter-kernel dependencies	No	Partial[a]	Yes	Partial[b]	Yes
Hierarchical heterogeneous compute (bank / chip / rank)	No	No	No	Partial[c]	Yes
Tensor placement and address interleaving awareness	No	Partial[d]	No	Partial[e]	Yes
Hierarchical, non-uniform communication modeling	No	Partial[f]	Yes	Partial[c]	Yes
Fast design space exploration with kernel-level statistics	Partial	Partial	Yes	Partial	Yes

[a] Graph execution is supported, but requires hardware-specific ONNX execution providers.
[b] Supports hierarchical placement or execution models, but not full end-to-end LLM inference DAGs.
[c] Hierarchy is modeled, but often assumes homogeneous banks or omits rank-/system-level heterogeneity.
[d] Placement or communication effects are modeled at an accelerator or NoC level, not with DRAM-internal address semantics.
[e] Placement abstractions exist, but integration with full LLM inference execution is limited.
[f] ONNXim uses BookSim as NoC model, which needs significant engineering effort to model non-uniform communication links.

or monolithic DRAM transactions. Conversely, system-level simulators emphasize communication across nodes but abstract away memory hierarchy details and tensor placement. As a result, none of these tools can natively express execution in which kernels are partitioned across bank-, chip-, and rank-level logic units, each with distinct access paths, contention points, and scheduling behavior.

Consequently, evaluating hierarchical PIM and PNM architectures for LLM inference requires a simulation framework with three key properties. First, it must capture end-to-end inference execution flow, including inter-kernel dependencies. Second, it must explicitly model hierarchical memory organization and tensor placement, rather than assuming uniform access costs. Third, it must support heterogeneous compute units at different memory levels and account for communication and queuing among them.

IV. Related Works

Prior simulators for ML accelerators typically optimize either (i) kernel-level systolic array modeling via GEMM traces, (ii) accelerator-centric graph simulation (e.g., ONNX-based) with shared-resource contention, or (iii) system-level execution-graph simulation (e.g., CHAKRA ET [41]). However, hierarchical PIM/PNM LLM inference requires simultaneously modeling end-to-end operator dependencies, tensor placement across the DRAM hierarchy, and communication and queuing overheads across heterogeneous logic units. Existing GEMM trace-driven kernel simulators do not encode inter-kernel dependencies required for LLM inference, limiting their ability to capture communication and serialization overheads. System-level DAG simulators scale to large systems but rely on coarse-grained, memory-agnostic abstractions that omit memory hierarchy and tensor placement, which are first-order concerns in PIM/PNM designs. ONNX-based approaches provide a standard front-end but require substantial engineering effort to implement an execution-provider for a target architecture, which is impractical when exploring emerging PIM/PNM designs. HARMONI substantially closes this gap by representing LLM inference as a task graph and explicitly modeling hierarchical memory systems with user-defined tensor allocation; it accounts for computation, communication, and queuing effects across heterogeneous logic units, enabling rapid design space exploration with kernel-wise and resource-wise statistics breakdown.

Tab. II compares existing simulator classes specifically with respect to their ability to model hierarchical PIM/PNM-based LLM inference as discussed in §III, rather than as general-purpose ML or DNN simulators. Many of these tools are highly effective within their intended abstraction scope (e.g., different data layout schemes in GEMM simulators, detailed micro-architecture modeling of a logic unit, high-level communication primitives modeling), but make design assumptions that limit direct applicability to memory-centric, hierarchical PIM/PNM architectures.

V. HARMONI Overview

HARMONI is a hierarchical architecture modeling framework designed to evaluate end-to-end LLM inference execution on near- and in-memory computing architectures. Rather than performing cycle-accurate simulation of individual components, HARMONI operates at a task-graph level, explicitly modeling memory accesses, computation, communication, and queuing delays across a configurable memory hierarchy.

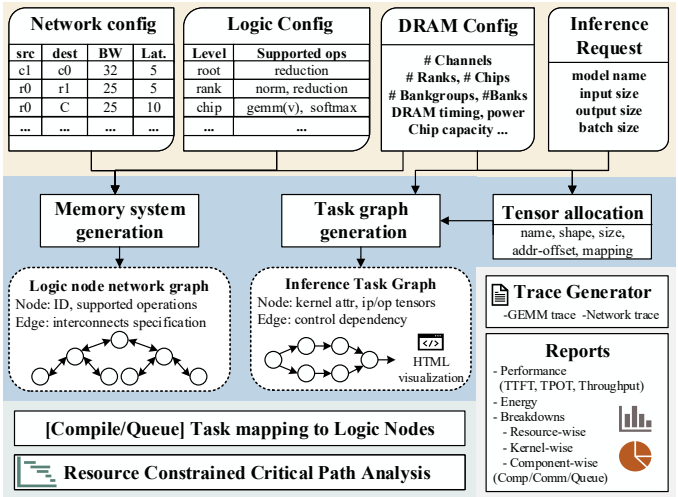

Fig. 3: HARMONI modeling framework

Fig. 3 provides an overview of the HARMONI execution flow. An inference request is transformed into a task graph whose nodes represent LLM kernels, and whose edges encode execution dependencies. This task graph is mapped onto a hierarchical architecture composed of logic units distributed across DRAM channels, ranks, chips, and banks. HARMONI then performs a resource-constrained critical path analysis to estimate end-to-end latency and energy.

The core design goal of HARMONI is to enable rapid design space exploration for memory-centric LLM architectures while preserving visibility into how tensor placement, memory organization, and logic placement jointly shape performance. The benefits of this abstraction are evaluated in §VI.

A. Input configurations

The architectural description is provided through three primary configuration inputs:

1) **DRAM configuration**, specifying the number of channels, ranks, chips, banks, chip capacity, GDL (Global Data Line) width, and power/timing characteristics,
2) **Logic configuration**, defining logic unit placement, supported operations, and area/power metrics obtainable from RTL synthesis, and
3) **Network configuration**, describing interconnect bandwidth, and latency between the logic nodes, and buffer size at intermediate nodes.

These inputs are independent of the LLM model and inference workload, allowing architectural parameters and DRAM configurations to be varied systematically.

B. Setup - Memory System Generation.

We define the memory system using DRAM, logic, and network configurations. Each logic unit is identified with an ID (i.e., the channel, rank, chip number). Logic units are specified per chip to support all-bank mode. The connection between the logic units are added based on the network topology. The PIM/PNM architecture (augmented with interconnects) shown in Fig. 2 and discussed in §III-A can be abstracted as a logic-node network graph, where the nodes correspond to logic-unit IDs and the edges represent interconnect links between nodes.

C. Application - LLM inference program representation

1) Memory Allocation for Tensors: The model configuration specifies hyperparameters such as $\#heads$, $\#layers$, and tensor dimensions, which are used to construct a dictionary of model weights ($Wqkv$, Wo, Wup, etc.). Each weight is represented by a custom tensor object with fields such as shape, size, address offset. These tensors are allocated in the memory system (§V-B). The allocation determines the address offset for each tensor. KV cache tensors can also be pre-allocated based on the batch size and maximum context length (batch-wise round-robin allocation to the "*kv-ranks*"). Each rank further reserves some memory for temporary tensors that are used to store intermediate results in the LLM inference program. All banks within a chip, and all chips within a rank, execute operations in lockstep to maximize aggregate DRAM bandwidth. Consequently, the effective memory access granularity is $\#chips \times \#banks \times GDL$.

The framework models DRAM address mapping using a configurable interleaving pattern that specifies the ordering of address fields, e.g., `Ch Ra Ro Co Bg Ba Bt`. Let the interleaving be $\mathcal{I} = [c_0, c_1, \ldots, c_n]$, where each component c_i (channel, rank, row, column, etc.) has an associated maximum value $\max(c_i)$. The bit-width for each component is

$$b_{c_i} = \lceil \log_2 \max(c_i) \rceil .$$

Given a request index, the address offset is constructed by concatenating these fields according to the interleaving pattern. If v_{c_i} is the value of component c_i extracted from the request index, then the final DRAM address-offset is

$$\text{addr_offset} = \sum_{i=0}^{n} (v_{c_i} \ll S_{c_i}), \qquad S_{c_i} = \sum_{j<i} b_{c_j}$$

where S_{c_i} is the cumulative shift contributed by preceding components in the interleaving sequence. Thus, the address encoding is fully determined by the interleaving pattern. In practice, HARMONI treats rank bits differently due to the separation of *kv-rank* and *weight-rank*; this simplified formulation omits that detail.

2) Task Graph Creation: This step acts as a graph-based execution trace for LLM inference. Each kernel (e.g., QKV projection, activation, head-wise attention) is modeled as a node in a directed task graph, and edges capture data dependencies and tensor movement. Each node is annotated

Algorithm 1: Tensor-Aware Task Mapping

Input: Task Graph G, Logic Network N, Tensor Map M
Output: Mapped Task Graph G'

```
G' ← G;
foreach node T ∈ G do
    Identify input tensors I_list and output tensors O_list;
    Retrieve partition sets P_in and P_out from M;
    if |P_in| > 1 then
        // partitioned across chips
        Split T into sub-tasks {t_1, ..., t_n} for each location;
        Insert aggregation node A to merge results;
        Place A at nearest common ancestor in N;
    else
        Assign T to a logic unit LU nearest (minimum hop) to
          location P_in that supports T;
    Update P_out in M based on task placement;
    Update dependencies and data movement edges in G';
return G';
```

with metadata: kernel type, model phase (prefill or decode), input/output tensors, etc. This metadata in the nodes helps with task mapping to logic nodes and statistics collection. The task graph is created by considering all the kernels in the decoding layer of any given model and iterating over it for each new token generated.

D. Compilation - Task Mapping to Logic Nodes

In this phase, each task node in the task graph is mapped to a logic unit in the memory system network as explained in Alg. 1. The mapping is determined by two factors: First, identifying the logic unit ID matching the tensor partitions. Second, verifying that the logic unit supports the task kernel. The mapping algorithm minimizes tensor movement across chips (by using weight and KV tensor stationary strategy), so only the intermediate output tensors move on the logic node network. If a kernel requires data from multiple DRAM partitions (e.g., different channels or ranks), it is split into sub-tasks, and an aggregation node is introduced and mapped to the nearest common parent logic node. The task graph is updated accordingly to reflect these decompositions. Each logic unit in the memory system network maintains a queue of tasks assigned to it. Similarly, the mapped logic unit per task is also added to the task graph.

E. Simulation

1) Execution Time Estimation: Finally, once the task graph is updated with the mapping, each node in the task graph has an assigned logic unit. The task graph with updated attributes is traversed to estimate the execution time for each kernel and communication time along graph edges. The execution time includes the time spent on the computation in the logic units and also the time spent on accessing data from the memory banks. We estimate DRAM access latency analytically based on the number of row activations and column accesses generated per tensor partition at each logic node, as shown in Eq. (1).

$$t = t_{CK} * (t_{CCD} * \#col_{access} + t_{RC} * \#row_{access}) \quad (1)$$

Since we have disaggregated execution on *wt-rank* and *kv-rank*, only one type of rank will be active at a time, so we use the idle time per rank to issue refresh commands.

2) Communication Cost Estimation: HARMONI models inter-node communication latency by extending the classical Hockney α-β model [42] to multi-hop, non-uniform hierarchical topologies with store-and-forward switching. The α term generalizes to per-hop static costs, the β term to collective-aware serialization via ϕ_k, and a buffer delay term that accounts for stalls when fan-in payload at an intermediate switch node exceeds its buffer capacity:

$$T(s,d) = \underbrace{L_0 + \sum_{k\in\mathcal{K}} (\ell_k + o_k)}_{\alpha} + \underbrace{\sum_{k\in\mathcal{K}} \frac{\phi_k \cdot S'}{B_k}}_{\beta} + \underbrace{\sum_{n\in\mathcal{N}} \max\left(0,\ \frac{F_n \cdot \phi_n \cdot S' - Q_n}{B_{n-1}}\right)}_{\text{buffer delay}} \quad (2)$$

where L_0 is endpoint latency; ℓ_k, o_k, B_k are propagation latency, router overhead, and bandwidth at hop k; F_n, Q_n are the fanout and buffer capacity at intermediate node n; S' is the flit-aligned message size; and ϕ_k is a collective-specific payload scaling factor. $\mathcal{K}=[d, s-1]$ for upward transfers (Gather, Reduce) and $[s, d-1]$ for downward transfers (Broadcast), with $\mathcal{N}$ the set of intermediate switch nodes. For Gather, ϕ_k grows as the product of path fanouts; for Reduce and all other operations, $\phi_k=1$. The buffer delay term follows D/D/1/K queuing [43] and vanishes for downward operations where $\mathcal{N}=\emptyset$. All parameters are obtained from the network configuration, and $T(s,d)$ yields $t_{\text{comm}}(p \to n)$ (Alg. 2).

3) Critical Path Analysis: To accurately estimate end-to-end inference latency, HARMONI employs a critical path analysis algorithm that accounts for data dependencies, communication overhead, and resource contention. In Alg. 2, we denote the earliest time at which all inputs of task n have arrived from its predecessors by $t_{\text{arrival}}(n)$, including communication latency. $t_{\text{ready}}(n)$ is the data-ready time for n, and $t_{LU}(n)$ is the earliest time at which the logic unit assigned to n (given by $LU(n)$) becomes free. The task start and finish times are $t_{\text{start}}(n)$ and $t_{\text{finish}}(n)$, respectively. $t_{\text{comm_overhead}}(n)$ captures the additional delay due to communication (beyond pure computation), and $t_{\text{stall}}(n)$ captures queuing delay due to logic-unit contention. $\text{preds}(n)$ is the set of predecessors of n in the task graph, and $\text{tasks}(LU)$ is the set of tasks mapped to logic unit LU. The final inference latency is determined by the t_{finish} of the last node in the critical path $\mathcal{P}_{\text{crit}}$.

F. Simulation Outputs

1) Statistics and Profiling Reports: HARMONI reports a core set of LLM inference metrics, including end-to-end

Algorithm 2: Critical-Path Analysis

Input : Task graph $G = (V, E)$;
execution time $t_{\text{exec}}(n)$ for each $n \in V$;
communication latency $t_{\text{comm}}(p \to n)$ for each $(p, n) \in E$;
task-to-logic-unit mapping $LU : V \to LU_set$
Output: Critical path $\mathcal{P}_{\text{crit}}$;

Forward pass (earliest times): foreach *task n in topological order of G* **do**
  if $\text{preds}(n) = \emptyset$ **then**
    $t_{\text{arrival}}(n) \leftarrow 0$;
    $\text{critical_parent}(n) \leftarrow \text{NIL}$;
  else
    $t_{\text{arrival}}(n) \leftarrow \max_{p \in \text{preds}(n)} \big(t_{\text{finish}}(p) + t_{\text{comm}}(p \to n)\big)$;
    $\text{critical_parent}(n) \leftarrow \arg\max_{p \in \text{preds}(n)} \big(t_{\text{finish}}(p) + t_{\text{comm}}(p \to n)\big)$;
  $t_{\text{ready}}(n) \leftarrow t_{\text{arrival}}(n)$;
  $t_{LU}(n) \leftarrow \max_{m \in \text{tasks}(LU(n))} t_{\text{finish}}(m)$;
  $t_{\text{start}}(n) \leftarrow \max\big(t_{\text{ready}}(n),\, t_{LU}(n)\big)$;
  $t_{\text{finish}}(n) \leftarrow t_{\text{start}}(n) + t_{\text{exec}}(n)$;
  $t_{\text{comm_overhead}}(n) \leftarrow t_{\text{arrival}}(n) - \max_{p \in \text{preds}(n)} t_{\text{finish}}(p)$;
  $t_{\text{stall}}(n) \leftarrow \max\big(0,\, t_{LU}(n) - t_{\text{ready}}(n)\big)$;

Backward pass (critical-path reconstruction):
$n_{\text{sink}} \leftarrow \arg\max_{n \in V} t_{\text{finish}}(n)$;
$\mathcal{P}_{\text{crit}} \leftarrow [\, n_{\text{sink}} \,]$;
$n \leftarrow n_{\text{sink}}$;
while $\text{critical_parent}(n) \neq \text{NIL}$ **do**
  $n \leftarrow \text{critical_parent}(n)$;
  prepend n to $\mathcal{P}_{\text{crit}}$;

(E2E) latency, Time to First Token (TTFT), Time Per Output Token (TPOT), decode tokens per second (TPS), total energy, and energy per token. E2E latency is further decomposed into computation, communication, and queuing components. HARMONI additionally records latency and energy for every kernel, tagged by operator type (e.g., GEMV, GEMM, RMSNORM), kernel category (e.g., Attention, Projection, and Embedding, and inference phase (Prefill or Decode) (see §V-C2). This phase-wise per-kernel breakdown identifies not only which kernels are expensive, but also which stages of the transformer pipeline dominate each inference phase on a given architecture.

Users can additionally generate task mapping statistics, resource-wise energy breakdowns, and per-node execution summaries for deeper architectural diagnosis.

2) Trace Generation: HARMONI can also generate traces for cycle-accurate backend modeling and post-processing to extract finer-grained execution statistics. Two trace categories are supported to capture compute and communication events.

GEMM traces emit per-chip kernel parameters containing the corresponding M, N, and K dimensions of each matrix multiplication after the task mapping phase (§V-D). These are compatible with SCALE-Sim [34] and can be adapted for other systolic-array simulators with appropriate modifications. When M=1, the resulting GEMV-style traces can be further processed to generate AiM-Simulator-compatible traces; however, accurate simulation additionally requires configuring the appropriate DDR timing parameters and data-mapping to match the target all-bank PIM architecture.

Network traces emit per-event communication entries derived from the task graph and memory system, capturing source, destination, message size, and communication operator type for each inter-node communication event. Although HARMONI models communication latency analytically using Eq. (2), these traces can be exported to network simulators such as BookSim [30] for higher-fidelity evaluation. However, BookSim's assumption of uniform network topologies and lack of native support for heterogeneous link bandwidths or reduction operations require source-code extensions to fully capture the configurations modeled in HARMONI.

VI. Evaluation

We evaluate HARMONI on the template hierarchical PIM/PNM architecture described in §III-A for various DRAM configurations (Tab. III). DRAM configurations are encoded as `STD-M#modules-R#ranksPerModule-C#chipsPerRank`, where `STD` is the memory standard (DDR4, DDR5, GDDR6). All the evaluated DDR configurations use 8 Gb chips. For instance, DDR5-M8-R4-C8 represents a DDR5 configuration with 8 modules, 4 ranks per module, 8 chips per rank. All configurations use `Ro Ch Ra Co Bg Ba Bt` address interleaving. GPU results use H100, and H100-SXM-2-80G latency measured via the vLLM framework [44]. Tab. III summarizes the design space parameters used in this study. In total, we conducted 168 simulations spanning three LLM models, four input/output token length combinations, three batch sizes, and 45 DRAM configurations covering DDR4, DDR5, and GDDR6. Not all combinations were simulated; instead, we selected representative configurations that isolate the effect of each architectural dimension. Figures report a subset of these results chosen to illustrate dominant trends.

A. Comparison with GPU

Fig. 4 compares E2E latency on the template PIM/PNM architecture for DDR5 configurations against GPU baselines; we make the following observations.

TABLE III: Design Space Parameters

Category	Configuration
LLM Models	Llama 2-7B [45], Llama 3-70B [46], Mistral 7B [47]
Datasets	In: 128, 2048, Out: 128, 2048, B: 1, 4, 8
GPUs	NVIDIA H100, H100-SXM-2-80G (referred as H100-2) [48]
DRAM Configs	Memory Standards: DDR4, DDR5, GDDR6 Chips (8Gb) (C): {4, 8, 16} Ranks (R): {2, 4, 8} Modules (M): {1, 2, 4, 8, 16}

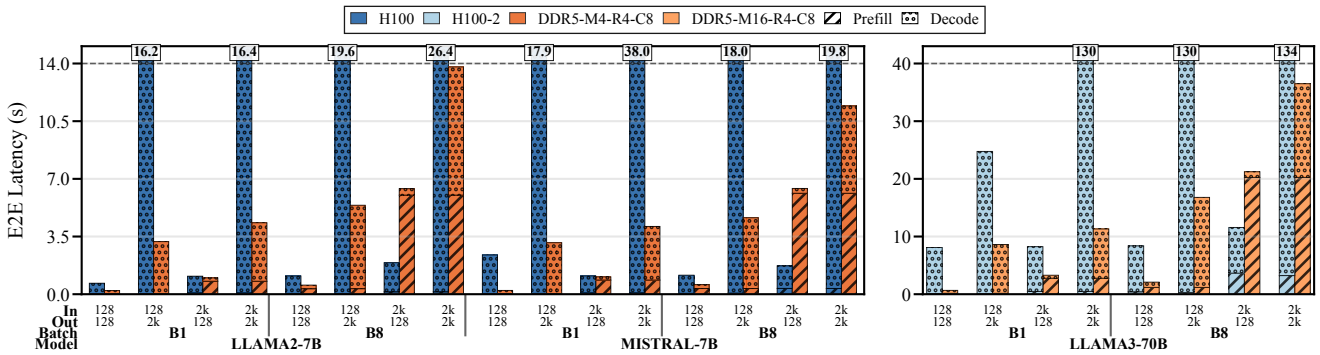

Fig. 4: End-to-End latency with prefill/decode breakdown across LLM models, batch sizes, and input/output configurations. Dashed horizontal lines indicate y-axis clipping; bars above the clip are annotated with their true E2E latency values.

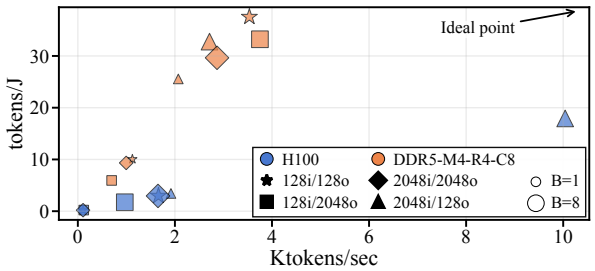

Fig. 5: Throughput vs. energy efficiency for Mistral 7B on H100 and DDR5-M4-R4-C8 across inference workloads.

Decode-heavy workloads (128i/2048o). DDR5-M4-R4-C8 outperforms H100 by 5.1× and 5.7× on Llama 2-7B and Mistral 7B respectively at B=1, as autoregressive decoding is bandwidth-bound and directly benefits from memory-centric execution. For Llama 3-70B, DDR5-M16-R4-C8 achieves a 3× speedup over H100-2, demonstrating that the PIM performance advantage persists at larger model scales despite the stronger GPU baseline.

Prefill-heavy workloads (2048i/128o). The advantage narrows to 1.08–1.11× across 7B models, since prefill is GEMM-dominated and PIM logic is compute-limited relative to H100. For Llama 2-7B, H100 completes prefill in 75 ms compared to 783 ms on DDR5-M4-R4-C8 at B=1.

Phase dominance. Despite slower prefill, DDR5-M4-R4-C8 achieves 3.8× lower E2E latency than H100 on the balanced 2048i/2048o case on Llama 2-7B at B=1, because decode dominates H100 execution (99.4% of E2E). On DDR5-M4-R4-C8, prefill accounts for a larger but still secondary fraction of E2E (18.1%), reflecting the reduced compute throughput of DDR5 PIM/PNM. This phase inversion is consistent across all evaluated workloads.

Impact of batch size. E2E latency increases with batch size on both H100 and DDR5 configurations. The increase is more pronounced on DDR5-M4-R4-C8 and DDR5-M16-R4-C8 due to higher queuing delay across logic nodes, as additional tasks compete for shared logic node resources.

Throughput vs. energy efficiency. Fig. 5 extends this comparison to the throughput-efficiency plane for Mistral 7B across batch sizes (B=1, B=8). Throughput and energy efficiency are measured over the end-to-end inference request, accounting for both prefill and decode tokens, computed as $(L_{in} + L_{out}) \times B$ normalized by total latency and total energy respectively. DDR5-M4-R4-C8 achieves markedly higher energy efficiency than H100 across all evaluated workloads: for decode-heavy configurations (128i/2048o), DDR5-M4-R4-C8 delivers 27× and 19× higher tokens/J than H100 at B=1 and B=8 respectively, as memory-centric execution avoids the high static power draw of GPU hardware. H100 achieves higher raw throughput only for prefill-heavy workloads (2048i/128o), where its tensor core advantage translates to a throughput gain at the cost of energy efficiency. Increasing batch size from B=1 to B=8 improves throughput on both systems while partially closing the efficiency gap, though DDR5-M4-R4-C8 retains a strong advantage. This highlights a fundamental design tradeoff: DDR5-M4-R4-C8 offers a consistently favorable energy-efficiency operating point across workloads, while H100 delivers higher throughput for compute-intensive prefill at significantly higher energy cost.

B. Sensitivity Study

We demonstrate HARMONI's utility for design space exploration across three axes: scale-out (number of DRAM modules), scale-up (number of ranks and chips per module), and memory standard selection.

1) Scale-Out Study: Fig. 6 shows E2E latency with computation, communication, and queuing breakdown for Llama 2-7B as the number of DDR5 modules increases from M1 to M8 across inference workloads.

Latency reduction. Scaling from M1 to M8 reduces E2E latency by 74–83% across all evaluated configurations, driven by improved memory-level parallelism and distribution of compute across more logic units.

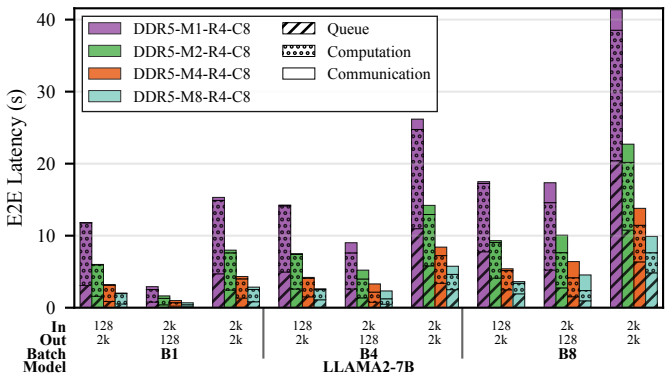

Fig. 6: ScaleOut: E2E breakdown vs. #DRAM modules

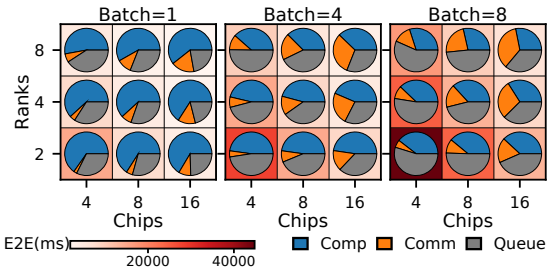

Fig. 7: ScaleUp: E2E breakdown vs. {#ranks, #chips} of DDR5 configs on Llama 2-7B for 2048i/2048o

Breakdown shift. The compute share of E2E drops consistently with scale-out (average 58.3% at M1 $\rightarrow$ 44.0% at M8), while the communication share rises (average 7.0% $\rightarrow$ 22.5%). This rise is not driven by growing communication cost, absolute communication time actually decreases slightly ($\sim$ 1.07 s at M1 vs. 0.85 s at M8), but by compute and queue time dropping faster. Communication does not become the bottleneck within the evaluated scale-out range.

Impact of batch size. Latency increases with batch size for all configurations, with the strongest sensitivity for prefill-heavy workloads (2048i/128o, $\sim 6\times$ from $B{=}1$ to $B{=}8$) and weakest for decode-heavy workloads (128i/2048o, 1.5–1.8$\times$). The bottleneck also shifts: queuing dominates at high batch for decode-heavy workloads (up to 53% of E2E), while communication dominates for prefill-heavy workloads at high scale and batch (45–48% at M8, $B{=}8$).

2) Scale-Up Study: Fig. 7 shows E2E latency as a heatmap over chips (C $\in \{4, 8, 16\}$) and ranks (R $\in \{2, 4, 8\}$) for Llama 2-7B at 2048i/2048o, with pie charts showing the computation, communication, and queuing breakdown per configuration.

Latency reduction. E2E latency decreases monotonically with both increasing chips and increasing ranks across all batch sizes. The heatmap gradient is steeper along the chip axis than the rank axis, reflecting a stronger per-step gain from chip scaling: the average speedup per chip doubling is 1.74$\times$ (C4$\rightarrow$C8) and 1.58$\times$ (C8$\rightarrow$C16), versus 1.67$\times$ (R2$\rightarrow$R4) and 1.45$\times$ (R4$\rightarrow$R8) per rank doubling, with diminishing returns on both axes.

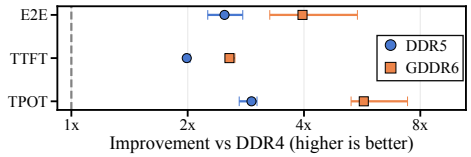

Fig. 8: Improvement in E2E latency, TTFT, and TPOT over DDR4 for DDR5 and GDDR6 (all M8-R4-C8). Markers show the median; whiskers show the min–max range.

Scaling efficiency. The best configuration (R8C16) outperforms the worst (R2C4) by 5.99$\times$ at $B{=}1$, 7.25$\times$ at $B{=}4$, and 6.44$\times$ at $B{=}8$. Since R8C16 represents a 16$\times$ resource increase over R2C4, the achieved speedup corresponds to 37–45% scaling efficiency, indicating that overheads (communication and queuing) consume a substantial fraction of the added resources.

Breakdown shift. At low-scale configurations, compute and queuing jointly dominate and communication is small. As scale increases, communication grows sharply: at B=8, moving from R2C4 to R8C16 shifts the breakdown from (comp 53.9%, comm 7.8%, queue 38.3%) to (comp 33.4%, comm 45.1%, queue 21.6%). Increasing chips with fixed ranks reduces queuing delay by distributing attention heads and FFN tasks across more logic nodes; increasing ranks with fixed chips reduces queuing caused due to batch-wise KV cache mapping across *kv-ranks*.

This suggests that smarter task-to-logic-unit mapping, such as load-balanced head placement or pipelined FFN dispatch, could recover further queuing delay beyond what architectural scaling alone provides. At extreme scale and high batch, communication exceeds compute, making kernel-only optimizations insufficient without corresponding reductions in collective overhead.

3) Impact of Memory Standards: Fig. 8 reports improvement in E2E latency, TTFT, and TPOT for DDR5-M8-R4-C8 and GDDR6-M8-R4-C8 relative to DDR4-M8-R4-C8 across all inference workloads (Tab. III) for Llama 2-7B and Mistral 7B.

The DDR5-over-DDR4 gain (2.48$\times$ geomean E2E gain, range 1.99–3.03$\times$) is attributable to the doubling of the bank count per chip (16 to 32) and the wider GDL (64-bit to 128-bit). The GDDR6-over-DDR5 gain (1.72$\times$ geomean, range 1.31–2.92$\times$) is driven primarily by the lower `tCCD` (1 ns vs. 2.5 ns), though GDDR6's reduced bank count (16 per chip) partially offsets its wider GDL (256-bit). The combined GDDR6-over-DDR4 gain reaches a 4.26$\times$ geomean (up to 8.84$\times$). The wider range for GDDR6 reflects stronger workload dependence: decode-heavy configurations benefit most from the `tCCD` advantage, while prefill-heavy cases show more modest gains. Empirically, performance is most sensitive to total bank count and per-bank bandwidth, with

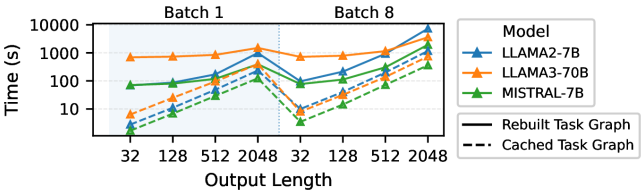

Fig. 9: HARMONI simulation wall-clock time

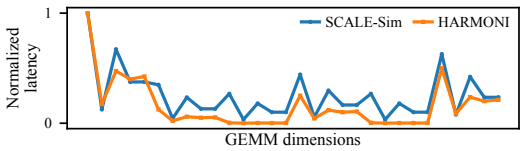

Fig. 10: SCALE-Sim raw compute cycles vs. HARMONI's compute cycles for 32 systolic arrays of size 8×8

memory standards ranking DDR4 < DDR5 < GDDR6 across all three metrics, consistent with their peak bandwidths.

C. HARMONI Simulation time and Optimizations

Fig. 9 shows HARMONI simulation wall-clock time on a single CPU as a function of output length and batch size. Runtime scales with model size and context length due to the increasing task-graph size, reaching a few hours for large models at long contexts (>2k tokens). For request sizes typical of ShareGPT or Alpaca datasets (a few hundred tokens), even uncached runs complete in seconds to minutes. **Graph caching.** The dominant cost in an uncached run is task-to-logic-unit mapping. Since mapping depends only on memory allocation, HARMONI caches the mapped task graph and reuses it across computational and communication design space sweeps, reducing simulation time by 2–3 orders of magnitude. **Single-layer reuse.** Since all decoder layers are structurally identical, HARMONI computes execution time for one layer and reuses it across the full stack. Tensor allocation is still performed for all layers to ensure accurate modeling of memory capacity and placement.

VII. Validation

PIM compute validation against AiM-Simulator. We validate HARMONI's bank-level compute and DRAM access timing against AiM-Simulator [16], [20]. We extract distinct GEMV kernel shapes from the Llama 2-7B and Mistral 7B inference task graphs and configure AiM-Simulator to model a single channel, matching one chip-level logic unit in HARMONI, under the same GDDR6 configuration. HARMONI achieves a Pearson correlation coefficient of r=0.99 and a mean absolute percentage error (MAPE) of 19.4%. The residual error is attributable to global buffer read/write overheads and the DRAM timing parameters not captured by HARMONI's analytical model (Eq. (1)).

Communication validation against BookSim. We validate the analytical communication model (Eq. (2)) against BookSim [30] on a hierarchical tree topology with uniform bandwidth. We evaluate all transfer sizes observed across two representative configurations: Llama 3-70B (128i/128o/B=1) and Llama 2-7B (128i/2048o/B=8), yielding 10 unique transfer sizes across 12,546,784 total transfer events. Of these, 12,544,784 are 1-byte fine-grained exchanges and approximately 2,000 are large KB–MB transfers, reflecting the highly skewed distribution characteristic of LLM inference communication. HARMONI achieves a MAPE of 24.1% overall: 1-byte transfers are over-predicted by 24.1% due to sub-flit granularity rounding in the static latency terms, while large transfers are under-predicted by 23.4% due to micro-architectural pipeline behavior not captured by the D/D/1/K store-and-forward term in Eq. (2). Validation of non-uniform bandwidth configurations with BookSim requires source-code extension and remains future work.

GEMM validation against SCALE-Sim. We evaluate GEMM kernels drawn from Llama 2-7B across varying input and batch dimensions, with $M \in [4, 1024]$, modeling 32 systolic arrays of size 8×8 under input-stationary dataflow and comparing against SCALE-Sim [34]. HARMONI achieves a Pearson correlation coefficient of r=0.90 and reproduces the same latency trend as SCALE-Sim (Fig. 10).

VIII. Discussion

HARMONI's analysis and visibility into LLM inference execution points to several opportunities for architectural and system-level innovation. Tensor-aware mappings that minimize data movement can introduce workload imbalance – alternating idle *weight-ranks* and *kv-ranks* and chip-wise attention-head placement (heads>chips) forces serialization, exposing opportunities for pipelining and compute–communication overlap. The bottleneck shifts across scale-out and scale-up configurations further suggest that mapping and network-level optimizations may be worth exploring. HARMONI currently targets static weight placement and task mapping; dynamic batching/scheduling and integration with tensor compilation flows are planned for future work.

IX. Conclusions

HARMONI is a fast, modular performance modeling framework for end-to-end LLM inference on hierarchical PIM/PNM architectures. It models computation, communication, and queuing across heterogeneous logic nodes, providing kernel-wise latency and energy breakdowns for systematic design space exploration. Evaluations across DRAM standards and scale-up/scale-out settings demonstrate its utility for PIM/PNM architecture research.

Acknowledgments. This work was supported in part by PRISM, one of seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA.

REFERENCES

[1] H. Li, Y. Li, A. Tian, T. Tang, Z. Xu, X. Chen, N. Hu, W. Dong, Q. Li, and L. Chen, "A survey on large language model acceleration based on kv cache management," *arXiv preprint arXiv:2412.19442*, 2024.

[2] Y. Xu, Z. Jie, H. Dong, L. Wang, X. Lu, A. Zhou, A. Saha, C. Xiong, and D. Sahoo, "Think: Thinner key cache by query-driven pruning," *arXiv preprint arXiv:2407.21018*, 2024.

[3] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao, "GEAR: An efficient kv cache compression recipe for near-lossless generative inference of llm," 2024.

[4] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.

[5] M. Adnan, A. Arunkumar, G. Jain, P. J. Nair, I. Soloveychik, and P. Kamath, "Keyformer: KV cache reduction through key tokens selection for efficient generative inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 114–127, 2024.

[6] P. G. Recasens, F. Agullo, Y. Zhu, C. Wang, E. K. Lee, O. Tardieu, J. Torres, and J. L. Berral, "Mind the memory gap: Unveiling GPU bottlenecks in large-batch LLM inference," *arXiv preprint arXiv:2503.08311*, 2025.

[7] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, 2024, pp. 117–134.

[8] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.

[9] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132.

[10] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "AttAcc! unleashing the power of PIM for batched transformer-based generative model inference," in *Proc. 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 2*, 2024, pp. 103–119.

[11] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, "Duplex: A device for Large Language Models with mixture of experts, grouped query attention, and continuous batching," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1429–1443.

[12] L. Liu, S. Zhao, B. Li, H. Ren, Z. Xu, M. Wang, X. Li, Y. Han, and Y. Wang, "Make LLM inference affordable to everyone: Augmenting GPU memory with NDP-DIMM," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1751–1765.

[13] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "PAPI: Exploiting dynamic parallelism in large language model decoding with a processing-in-memory-enabled computing system," in *Proc. 30th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 2*, 2025, pp. 766–782.

[14] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, "SpecPIM: Accelerating speculative inference on PIM-enabled system via architecture-dataflow co-exploration," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 950–965.

[15] B. Y. Cho, J. Jung, and M. Erez, "Accelerating bandwidth-bound deep learning inference with main-memory accelerators," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–14.

[16] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "PIM is all you need: A CXL-enabled GPU-Free system for Large Language Model inference," in *Proc. 30th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 2*, 2025, pp. 862–881.

[17] M. Zhou, W. Xu, J. Kang, and T. Rosing, "TransPIM: A memory-based acceleration via software-hardware co-design for transformer," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 1071–1085.

[18] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing," in *Proc. 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3*, 2024, pp. 722–737.

[19] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin *et al.*, "IANUS: Integrated accelerator based on NPU-PIM unified memory system," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 545–560.

[20] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, and J. Chun, "System architecture and software stack for GDDR6-AiM," in *2022 IEEE Hot Chips 34 Symposium (HCS)*, 2022, pp. 1–25.

[21] S.-S. Park, K. Kim, J. So, J. Jung, J. Lee, K. Woo, N. Kim, Y. Lee, H. Kim, Y. Kwon, J. Kim, J. Lee, Y. Cho, Y. Tai, J. Cho, H. Song, J. H. Ahn, and N. S. Kim, "An LPDDR-based CXL-PNM platform for TCO-efficient inference of transformer-based Large Language Models," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 970–982. [Online]. Available: https://ieeexplore.ieee.org/document/10476443/

[22] C. Ortega, Y. Falevoz, and R. Ayrignac, "PIM-AI: A novel architecture for high-efficiency LLM inference," *arXiv preprint arXiv:2411.17309*, 2024.

[23] W. Han, H. Cho, D. Kim, and J.-Y. Kim, "SAL-PIM: A subarray-level processing-in-memory architecture with LUT-based linear interpolation for transformer-based text generation," *IEEE Transactions on Computers*, 2025.

[24] L. Yan, X. Lu, X. Chen, Y. Han, and X.-H. Sun, "Pyramid: Accelerating LLM inference with cross-level processing-in-memory," *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 121–124, 2025.

[25] T. Jeong and E.-Y. Chung, "PipePIM: Maximizing computing unit utilization in ML-oriented digital PIM by pipelining and dual buffering," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 12, pp. 4585–4598, 2024.

[26] Y. Wu, Z. Wang, and W. D. Lu, "PIM GPT a hybrid process in memory accelerator for autoregressive transformers," *npj Unconventional Computing*, vol. 1, no. 1, p. 4, Jul. 2024. [Online]. Available: https://www.nature.com/articles/s44335-024-00004-2

[27] Y. Jang, H. Cho, Y. Ryu, J. Kim, and S. Hong, "PIMPAL: Accelerating LLM inference on edge devices via in-DRAM arithmetic lookup," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025, pp. 1–7.

[28] G. F. Oliveira, J. Gómez-Luna, S. Ghose, A. Boroumand, and O. Mutlu, "Accelerating neural network inference with processing-in-DRAM: from the edge to the cloud," *IEEE Micro*, vol. 42, no. 6, pp. 25–38, 2022.

[29] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "GARNET: A detailed on-chip network model inside a full-system simulator," in *Performance Analysis of Systems and Software, 2009. ISPASS 2009. IEEE International Symposium on*. IEEE, 2009, pp. 33–42.

[30] N. Jiang, D. U. Becker, G. Michelogiannakis, J. Balfour, B. Towles, D. E. Shaw, J. Kim, and W. J. Dally, "A detailed and flexible cycle-accurate network-on-chip simulator," in *2013 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2013, pp. 86–96.

[31] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.

[32] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible DRAM

simulator," *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, 2023.
[33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
[34] R. Raj, S. Banerjee, N. Chandra, Z. Wan, J. Tong, A. Samajdhar, and T. Krishna, "SCALE-Sim v3: A modular cycle-accurate systolic accelerator simulator for end-to-end system analysis," in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2025, pp. 186–200.
[35] M. Choudhary, C. Kjellqvist, J. Ma, and L. W. Wills, "COCOSSim: A cycle-accurate simulator for heterogeneous systolic array architectures," in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2025, pp. 174–185.
[36] H. Ham, W. Yang, Y. Shin, O. Woo, G. Heo, S. Lee, J. Park, and G. Kim, "ONNXim: A fast, cycle-level multi-core NPU simulator," *IEEE Computer Architecture Letters*, 2024.
[37] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-sim: Enabling SW/HW co-design exploration for distributed DL training platforms," in *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2020, pp. 81–92.
[38] J. Cho, M. Kim, H. Choi, G. Heo, and J. Park, "LLMServingSim: A HW/SW co-simulation infrastructure for LLM inference serving at scale," in *2024 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2024, pp. 15–29.
[39] F. A. Siddique, D. Guo, Z. Fan, M. Gholamrezaei, M. Baradaran, A. Ahmed, H. Abbot, K. Durrer, K. Nandagopal, E. Ermovick, K. Kiyawat, B. Gul, A. Mughrabi, A. Venkat, and K. Skadron, "Architectural modeling and benchmarking for digital DRAM PIM," in *2024 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2024, pp. 247–261.
[40] T. Xie, Z. Zhu, B. Li, Y. He, C. Li, G. Sun, H. Yang, Y. Xie, and Y. Wang, "UniNDP: A unified compilation and simulation tool for near DRAM processing architectures," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 624–640.
[41] S. Sridharan, T. Heo, L. Feng, Z. Wang, M. Bergeron, W. Fu, S. Zheng, B. Coutinho, S. Rashidi, C. Man *et al.*, "Chakra: Advancing performance benchmarking and co-design using standardized execution traces," *arXiv preprint arXiv:2305.14516*, 2023.
[42] R. W. Hockney and C. R. Jesshope, *Parallel Computers 2: architecture, programming and algorithms*. CRC Press, 2019.
[43] D. G. Kendall, "Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded markov chain," *The Annals of Mathematical Statistics*, pp. 338–354, 1953.
[44] W. Kwon, Z. Li, S. Zhuang, S. Shao, J. M. Gonzalez, R. Nishihara, I. Stoica, and K. Kasimov, "Efficient memory management for Large Language Model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
[45] "Llama-2-7b model page," https://huggingface.co/meta-llama/Llama-2-7b, 2023, meta AI, Hugging Face, accessed Mar. 17, 2026.
[46] "Meta-Llama-3-70B model page," https://huggingface.co/meta-llama/Meta-Llama-3-70B, 2024, meta AI, Hugging Face, accessed Mar. 17, 2026.
[47] "Mistral-7B-v0.1 model page," https://huggingface.co/mistralai/Mistral-7B-v0.1, 2023, mistral AI, Hugging Face, accessed Mar. 17, 2026.
[48] "NVIDIA H100 GPU Datasheet." [Online]. Available: https://nvdam.widen.net/s/fdllbtmmbv/h100-datasheet-2430615

APPENDIX

A. Abstract

HARMONI is a Python-based performance modeling framework that facilitates custom memory allocation (determine the location of each tensor to model memory access latency), inference program representation (acting as an intermediate representation in the form of a task graph accounting for all the kernels and dependencies among them), task mapping to the logic nodes (mimicking program compilation and task scheduling), and statistics collection (kernel-wise latency and energy breakdown for further analysis). This document provides instructions to reproduce the main results in the paper.

B. Artifact check-list (meta-information)

- **Run-time environment:** Linux (x86_64), Conda env, Python
- **Execution:** `run_ae.sh`, `analyse_ae.sh`
- **Metrics:** prefill latency, decode latency, E2E latency, decode throughput, total energy, compute/communication/queuing percentages
- **Output:** CSV and PDF files `results/*`
- **Experiments:** LLM inference performance modeling for 120 cases for different LLMs, In/Out/Batch sizes on various DRAM configurations. Parameters are listed in `params/ispass_ae_*.txt`
- **How much disk space required (approximately)?:** ~200GB
- **How much time is needed to prepare workflow (approximately)?:** 5-10 minutes
- **How much time is needed to complete experiments (approximately)?:** The simulation time varies for each individual experiment from 2 minutes to 20 hours. Experiments in this artifact take ~10 hours (with SLURM), or ~35 hours (with local machine having ~150GB RAM) for all the 120 cases.
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT License
- **Archived (provide DOI)?:** https://doi.org/10.5281/zenodo.19138546

C. Description

1) How to access: The source code, scripts to run simulations, analyze results, and instructions are publicly available on GitHub: https://github.com/UVA-LavaLab/HARMONI

2) Hardware dependencies: HARMONI can be run on any Linux x86_64 machine. The memory requirement of the machine depends on the simulation parameters (i.e., `dram`, `model`, and `in/out/batch`). Parameters for this artifact have been binned into 5 categories in `params/ispass_ae_*.txt` based on the maximum memory requirement for each bin.

Bin	small	mid	large	large1	large2
Max Mem (GB)	16	32	64	128	150

Full artifact reproduction of 120 cases is recommended on a SLURM-enabled CPU cluster for faster simulation.

3) Software dependencies: To run HARMONI, you need Conda, Python 3.10, and Python packages from `requirements.txt`.

D. Installation

Use the steps below to clone the HARMONI repository, create and activate a Conda environment, and install required packages.

```
>> git clone https://github.com/UVA-LavaLab/
   HARMONI.git
>> cd HARMONI
```

```
>> conda create -n harmoni -c conda-forge
    python=3.10 pip graphviz -y
>> conda activate harmoni
>> pip install -r requirements.txt
```

E. Experiment workflow

The code structure of HARMONI is mentioned in `README.md`. All commands should be run from the repository root (`HARMONI/`).

After the installation setup, run `env.sh` to activate harmoni conda environment, set `HARMONI_ROOT`, and create necessary directories.

```
>> source env.sh
```

Optionally clean up before a fresh run

```
>> source clean.sh
```

Different ways to run experiments:

Single Experiment for Trial: Use `run.sh` to run a single experiment by modifying it with the desired parameters and command line options or see "Usage" in `README.md`.

```
>> source run.sh
```

Combined Experiments for Artifact: Use one of the following ways to run the experiments in the artifact and reproduce the results. All the 120 cases used in this artifact are categorized and listed in `params/ispass_ae_*.txt`. The launch scripts read the parameters from these files and launch parallel jobs. **Edit `launch_ae_jobs.sh` to configure launch options for the selected method.**

1) SLURM Cluster (recommended): Proposed edits in `launch_ae_jobs.sh`:

- set `DEFAULT_LAUNCH_MODE` to `slurm`
- set `SLURM_EMAIL` and `SLURM_PARTITION`
- set `SLURM_ALLOCATION` (if required)
- optionally set `SLURM_ONLY` to one of the parameter bins (small, mid, large, large1, large2). *Set this **only if** you do not have enough disk space and/or nodes with required memory capacity*. [Recommended] Not setting this launches all the 120 jobs.

`launch_ae_jobs.sh` reads the template `slurm_ispass_ae_*.sh` provided in the repository and launches jobs after overriding some options. Run the following after making the edits:

```
>> source run_ae.sh
```

Wait for all jobs to finish. You can check the job status by using `squeue` or looking for job completion email. Verify there are no errors in `logs_slurm/`.

2) Local Machine (>150GB RAM): Proposed edits in `launch_ae_jobs.sh`:

- set `DEFAULT_LAUNCH_MODE` to `local`
- set `LOCAL_SIZES` to "" or comma-separated parameter bins. *Set this **only if** you do not have enough disk space.* [Recommended] Setting to "" launches all the 120 jobs.

```
>> source run_ae.sh
```

Wait for all jobs to finish.

F. Evaluation and expected results

Once all the jobs finish without errors, run the following script that analyses the output files and compares with reference results provided in `reference/` and plot figures in the paper.

```
>> source analyse_ae.sh
```

Expected results:

1) Simulation stage: should generate per-case files in `outputs/`:

- `performance_summary_*.txt`,
- `energy_breakdown_*.txt`,
- `critical_path_breakdown_*.txt`.

2) Analysis stage: should move aggregated files in `results/`, and create `results/hierarchical_data.csv`.

3) Expected pass message: `analyse_ae.sh` runs *diff_hier_data.py* that compare simulated results with a reference results collected from HARMONI. Look for: `[PASS] All relevant metrics matched the reference values.`

4) Expected figure outputs: `analyse_ae.sh` runs *scripts/figures/ispass.py* that generates Fig. 4, 5, 6, and 7 in the paper located at `results/plots/E2E_PD_breakdown_subplots.pdf`, `results/plots/mistral_scatter.pdf`, `results/plots/scale_out_breakdown.pdf`, and `results/plots/scaleup_heatmap_pies.pdf` respectively.

G. Experiment customization

- Running experiments for input/output/batch size other than the artifact: Create `params/*` with new set of parameters to test. Update the analysis scripts appropriately.
- Additional analysis: Add more options to dump stats or traces in the run.py command in `slurm_ispass_ae*` scripts or `run_all_params_memaware.sh`

H. Notes

- If you change task mapping logic, clear `graph_cache/` (uncomment rm command in `clean.sh`) before rerunning.
- `run_ae.sh` is interactive and asks for confirmations.

I. Methodology

Submission, reviewing and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-and-badging-current
- https://cTuning.org/ae